

Introduction to the Command Line and R Studio

Open Rstudio

Download from Lab Exercises “01_Getting_Started_in_R.Rmd” from OneDrive (check Blackboard Announcements for the link)

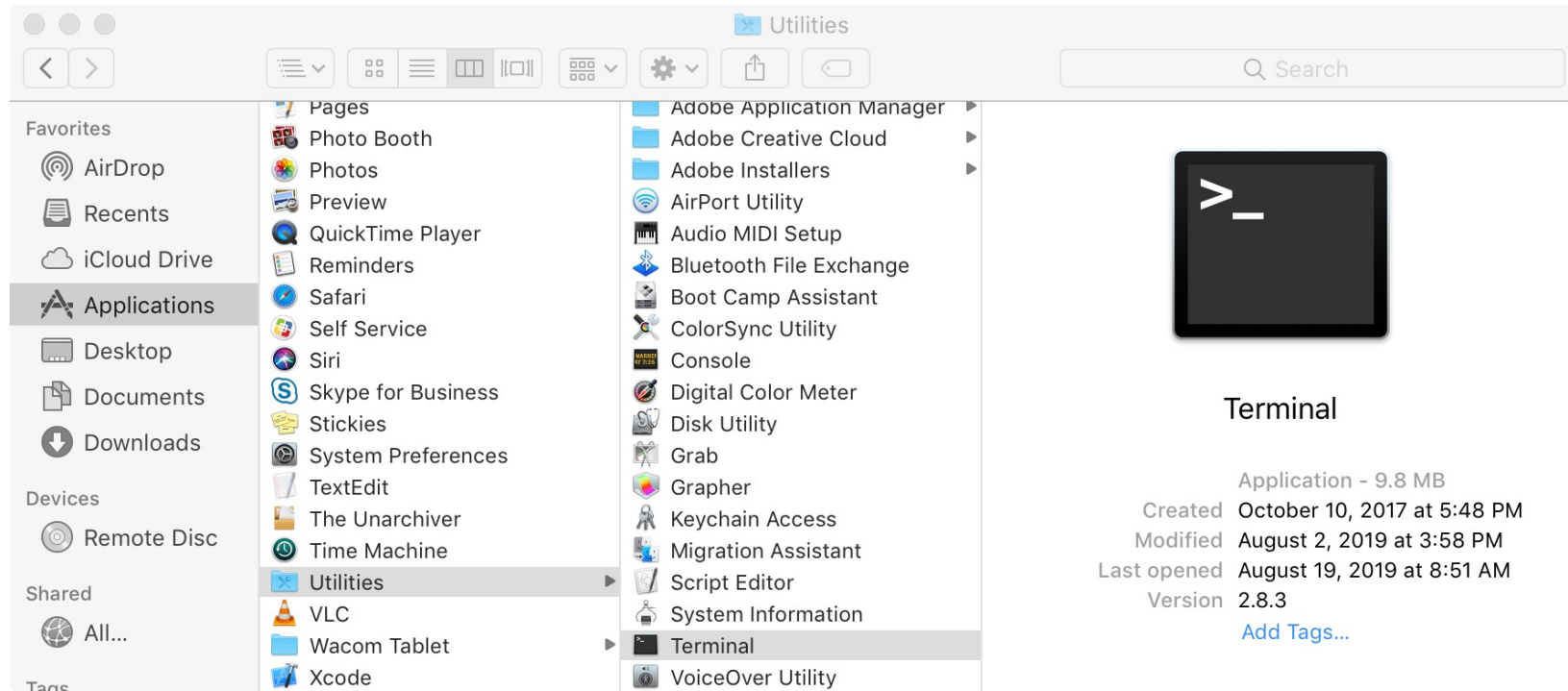
You should have received an email from DataCamp with a free membership for the semester

Reminder: fill out the request for AHPCC access—Andy Alverson will give a guest lecture on how to submit jobs (possibly the most useful thing you will learn in this course!)

WTF is the Command Line?

- The command line (interface) is a way to interact directly with your computer by typing in “commands” in the form of simple text
- Your computer operating system actually interfaces with the command line through a “command-line interpreter”
- Command line interpreters are also called “shells,” and these are essentially programs that read commands and then execute (or run) those commands
- A GUI (graphical user interface) is a type of interpreter that allows users to interact via icons (instead of using the text-based command line)
 - Windows and MacOS are examples of GUIs
- The terminal is a command line interface that allows users to directly type commands to execute

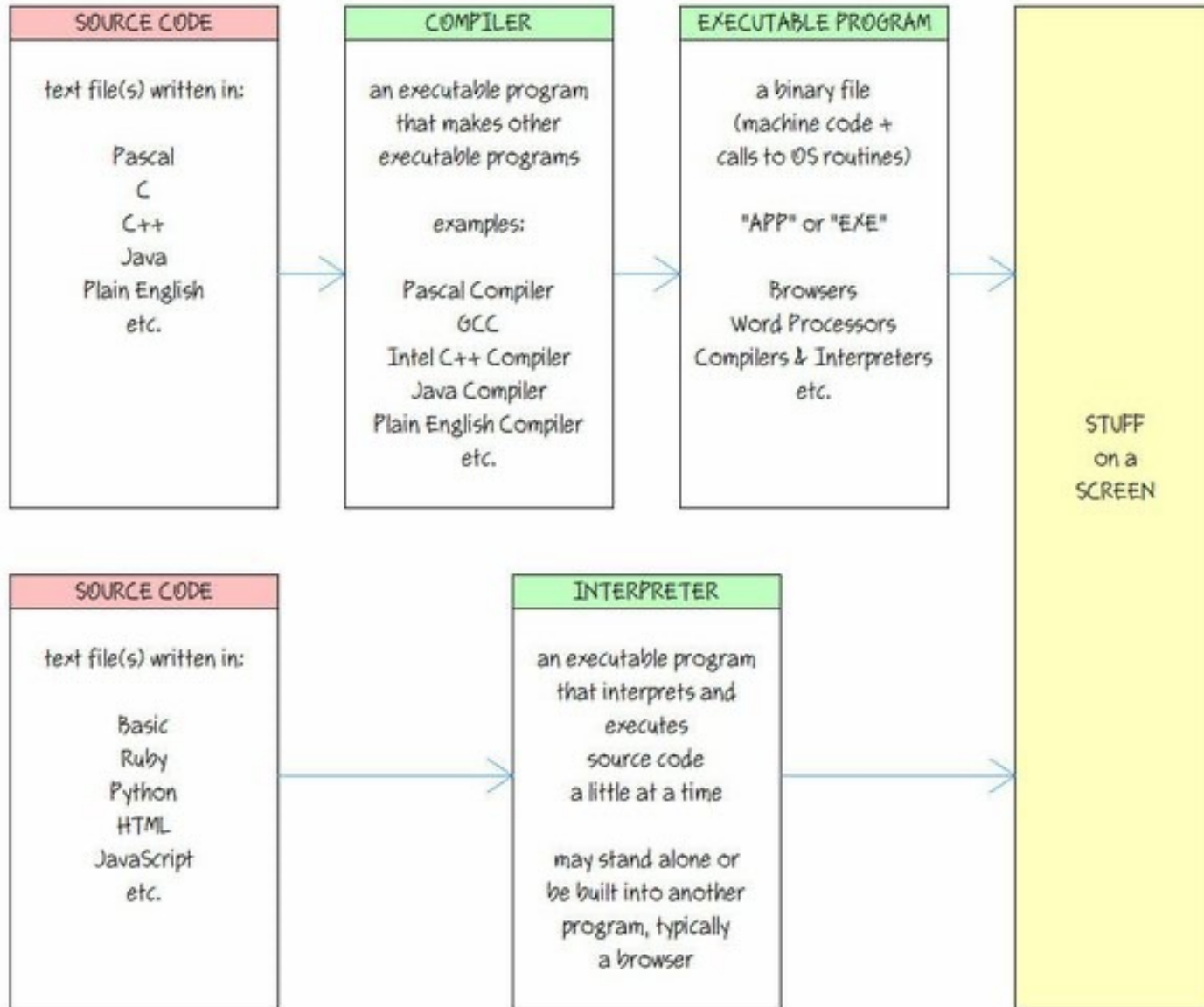
The Terminal



Programming Languages

- script: a program or set of commands that is carried out by another program rather than the computer processor
- There are several programming languages that are called scripting or interpretive languages, where the code is executed in one step
- Interpretive languages are commonly used in genomics because they work really well with text (e.g., lines of sequences or gene names)
 - Common ones include bash, python, R, ruby, perl (mostly replaced by python)
- In contrast, compiled languages are converted in a machine code (unreadable by humans) that is then executed by the processor
- Compiled programs run much, much faster than interpretive ones
 - Compiled languages include C++, Java, Rust, Julia

Programming Languages



Resources for getting unstuck with computational biology

- O'neil. A Primer for Computational Biology:
<https://open.oregonstate.edu/computationalbiology/>
- Cheat sheet of Unix commands:
- <https://www.hostinger.com/tutorials/linux-commands>
- <https://blog.invgate.com/cheat-sheet-for-unix-commands>

Everybody's First Command...

- The “echo” command in the bash command language displays text

type:

```
echo Hello World
```

R

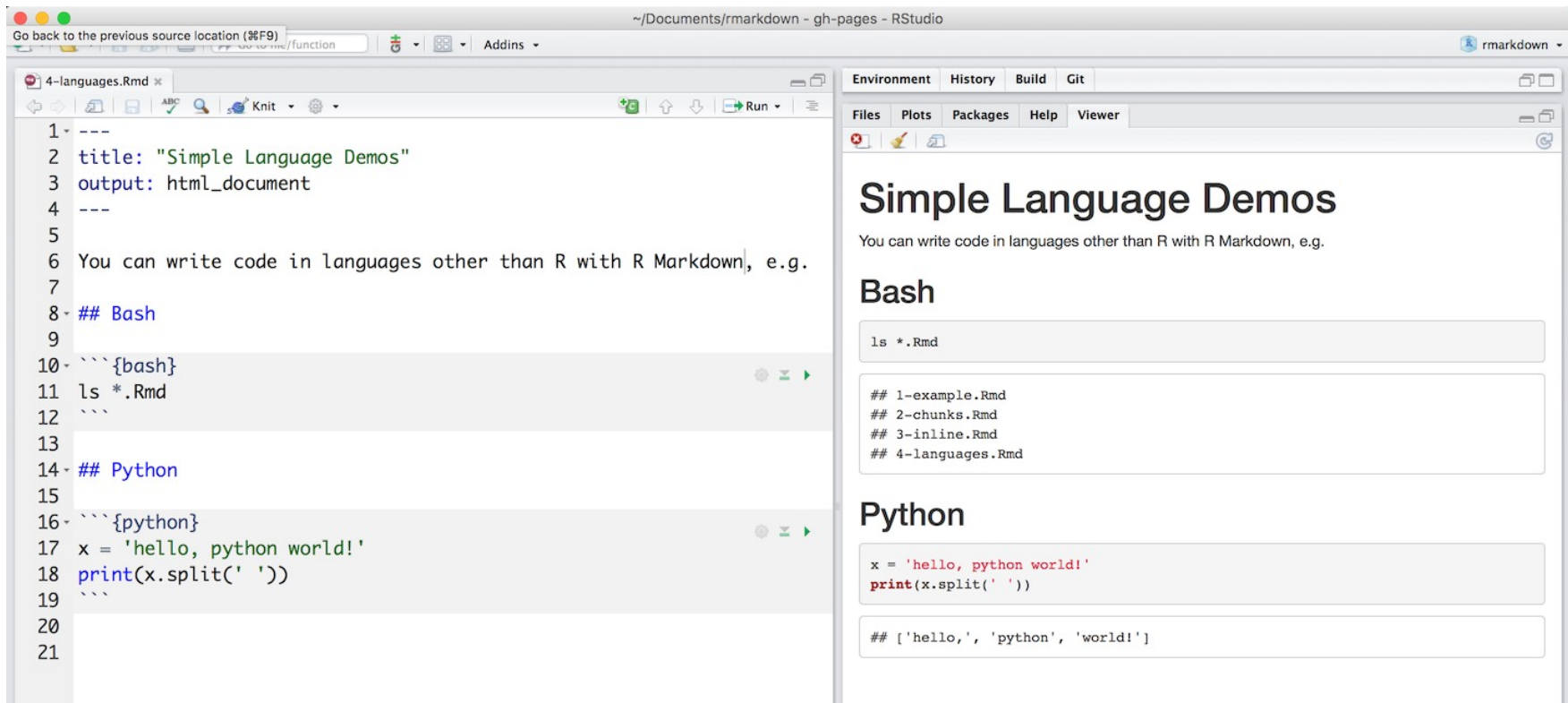
- R is a scripting language for statistics and graphics
- R “packages” are collections of code, data, and documentation that can be used for specific tasks
- The Comprehensive R Archive Network (CRAN) is a central repository for R packages (16,000 total as of 2020)
- Bioconductor is another repository for software packages geared towards bioinformatics and genomic analyses (2,200 packages as of 2023)
- R is widely used for genomic analysis because it's free, open-source, and rather powerful for statistical analysis of large datasets

RStudio

- an integrated development environment for R
- makes it much easier to use R
- supports R Markdown (your electronic lab notebook)
- RStudio Manual: <https://www2.stat.duke.edu/courses/Spring14/sta101.001/UsersGuide.pdf>
- RStudio Cheat Sheet: https://ucdavis-bioinformatics-training.github.io/Oct2017-ILRI-Workshop/Cheat_Sheets/rstudio-IDE-cheatsheet.pdf

R Markdown

- allows you to run your code (commands) in chunks
- can run not only R code, but also bash and python code



R Markdown

- allows you to run your code (commands) in chunks
- can run not only R code, but also bash and python code
- # is a symbol that tells R, bash, and python not to run that line of code
 - used to add “comments,” for example what that line of code does, or what the options mean
- Can output R Markdown files as html or pdf files (useful for sharing as supplements for your publications, for turning in homework assignments, and for working on your final project)

Reproducible (computational) research

- computational research is **reproducible** if someone else can take the data and code and produce the exact same set of results
- this is our minimum standard for performing and publishing computational research
- just because something is reproducible does not mean it is correct (the code can have bugs, the data may be poor quality, etc.)
- R Markdown is a great tool for reproducibility, because you can document all of the code used and have someone else run it to obtain identical results

Basic principles of reproducible research

- try not to do anything by hand (use code for all analyses)
- automate your workflow
- start with data in the rawest possible form (e.g., raw sequence files)
- scrutinize the metadata (defined as all of the information about how the data was collected) to know how the experiments were performed (if you did not collect the data yourself)
- compile accurate records of data collection and save as metadata (if you did collect the data yourself)
- keep track of all data files and analysis files that are generated

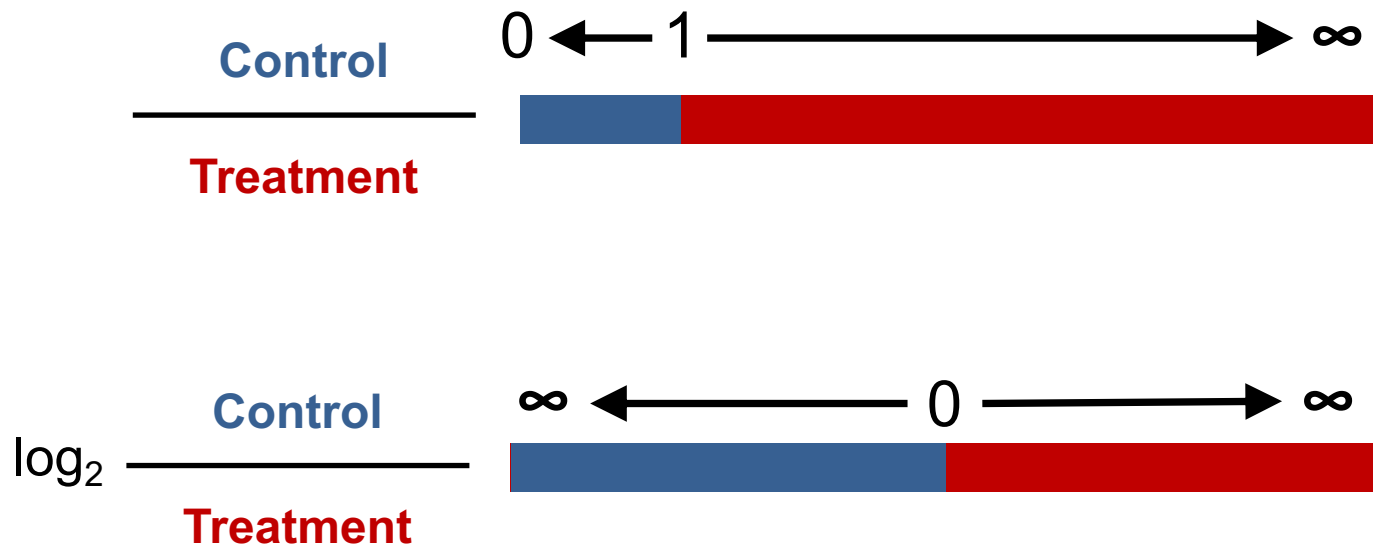
Okay, enough on the computing side—let's look at a biological example

Comparing relative gene expression between samples (RNA or protein)

We are frequently interested in ratios of gene expression:

- what genes have lower expression in the mutant vs wild type?
- what genes have higher expression in embryo vs adult?
- what genes have higher expression in response to stress?

Ratios are reported in $\log_2$ space (i.e., $\log_2$ fold changes)



Comparing relative gene expression between samples

To convert $\log_2$ ratios back to linear space, the formula is 2^n

Here's some good mental short-hand conversions between $\log_2$ and linear space:

$\log_2$ fold change

0

0.585

1

1.585

2

3

4

5

linear fold change

1

1.5 (less common cutoff)

2 (common cutoff)

3

4

8

16

32

Significance Testing

We'll spend a lot more time talking about significance testing, but you're about to see p -values and FDR-corrected p -values

Here's the short version for gene expression:

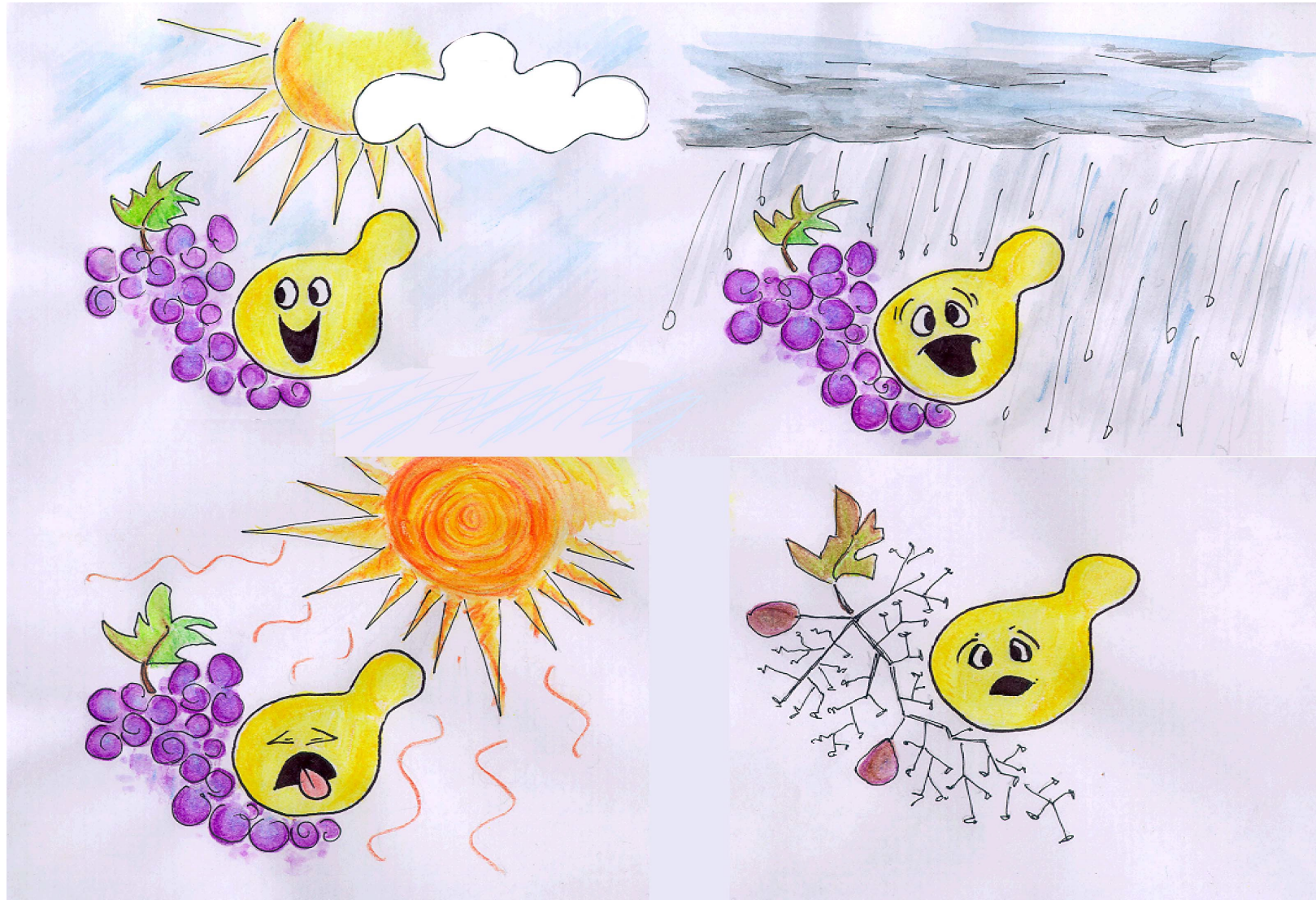
- The p -value for each gene is the the probability of obtaining $\log_2$ fold changes at least as extreme as those observed, under the assumption that the null hypothesis is correct (i.e., there is no actual difference $\log_2$ fold changes)
- However, gene expression analysis means that you are testing the expression of thousands of genes at once
- If you just use a p -value cutoff of 0.05 or even 0.01, you will have hundreds of false positives (what's 5% or 1% of 20,000 genes)
- The false discovery rate (FDR) allows you to control the number of false positives you allow (e.g., 1% relative to true positives)

Significance Testing

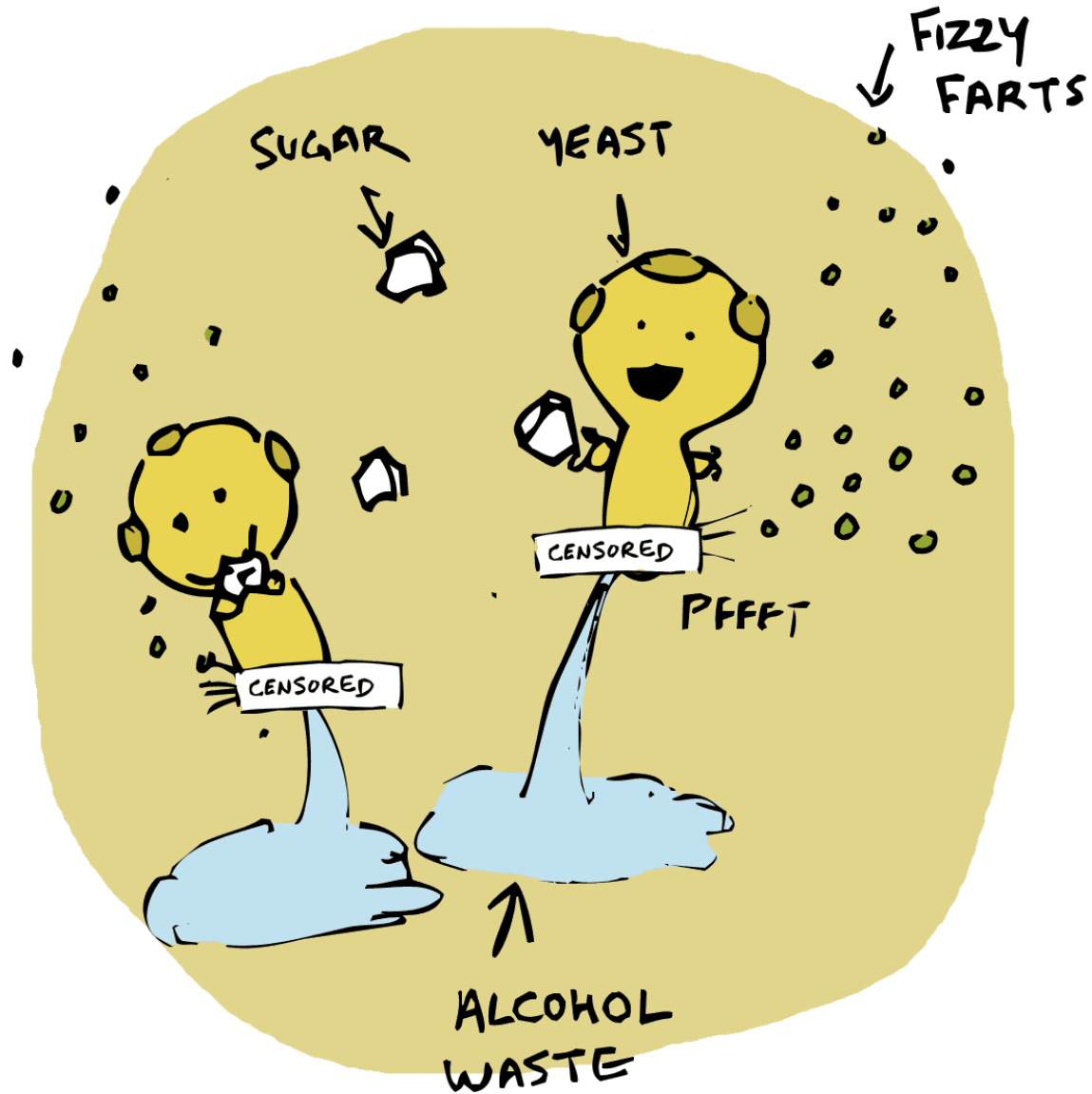
For example, if you have 99 true positives (genes with real differential expression) out of 20,000 total genes in the genome:

- An FDR threshold of 0.01 will yield 100 genes with differential expression (1 false positive and 99 true positives)
- Just a p -value threshold of 0.01 will yield 299 genes with differential expression (200 false positives and 99 true positives)
- This is a bit of an oversimplification, because we're ignoring false negatives and there are different ways to estimate FDR

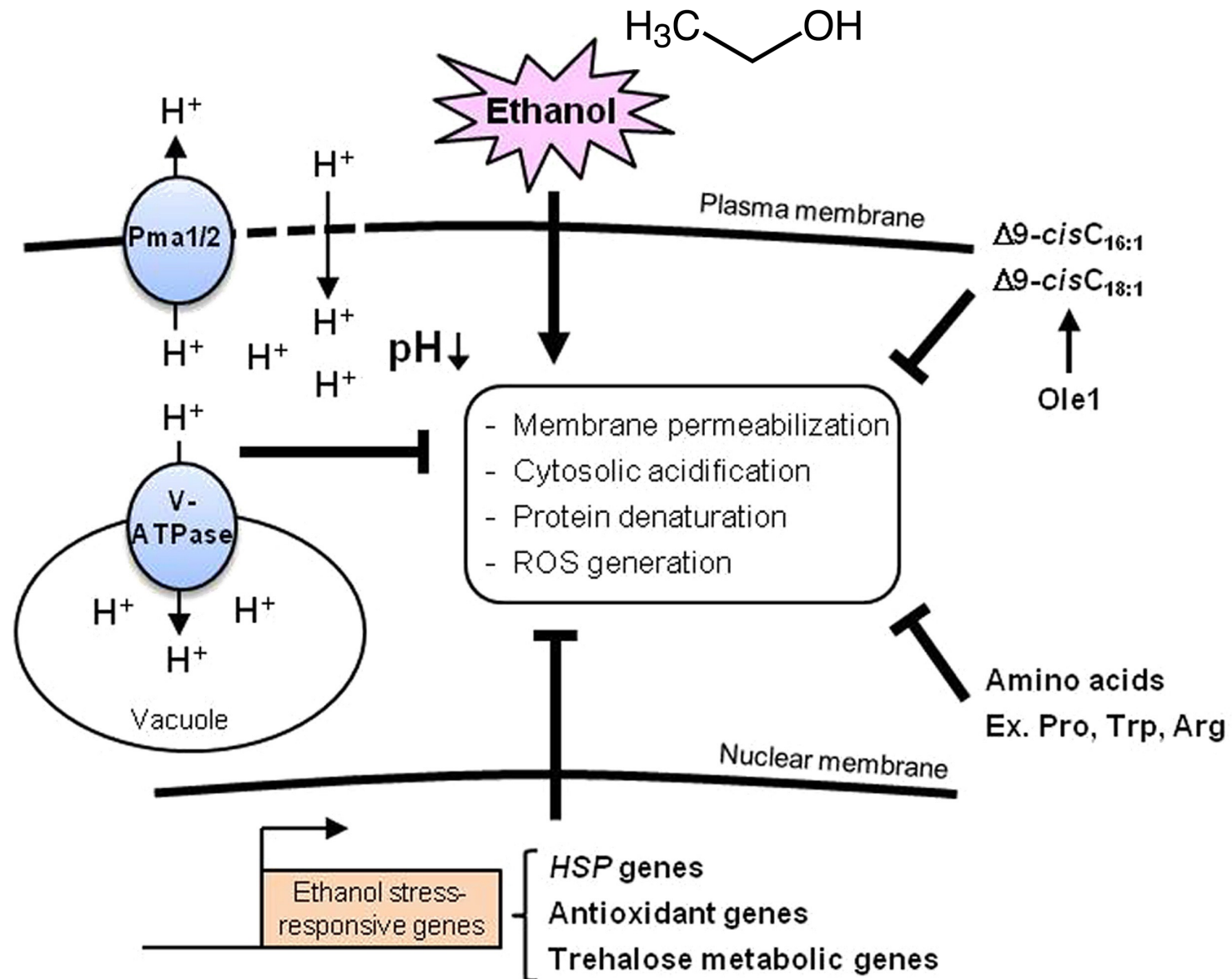
Biological example from my lab: yeast stress responses



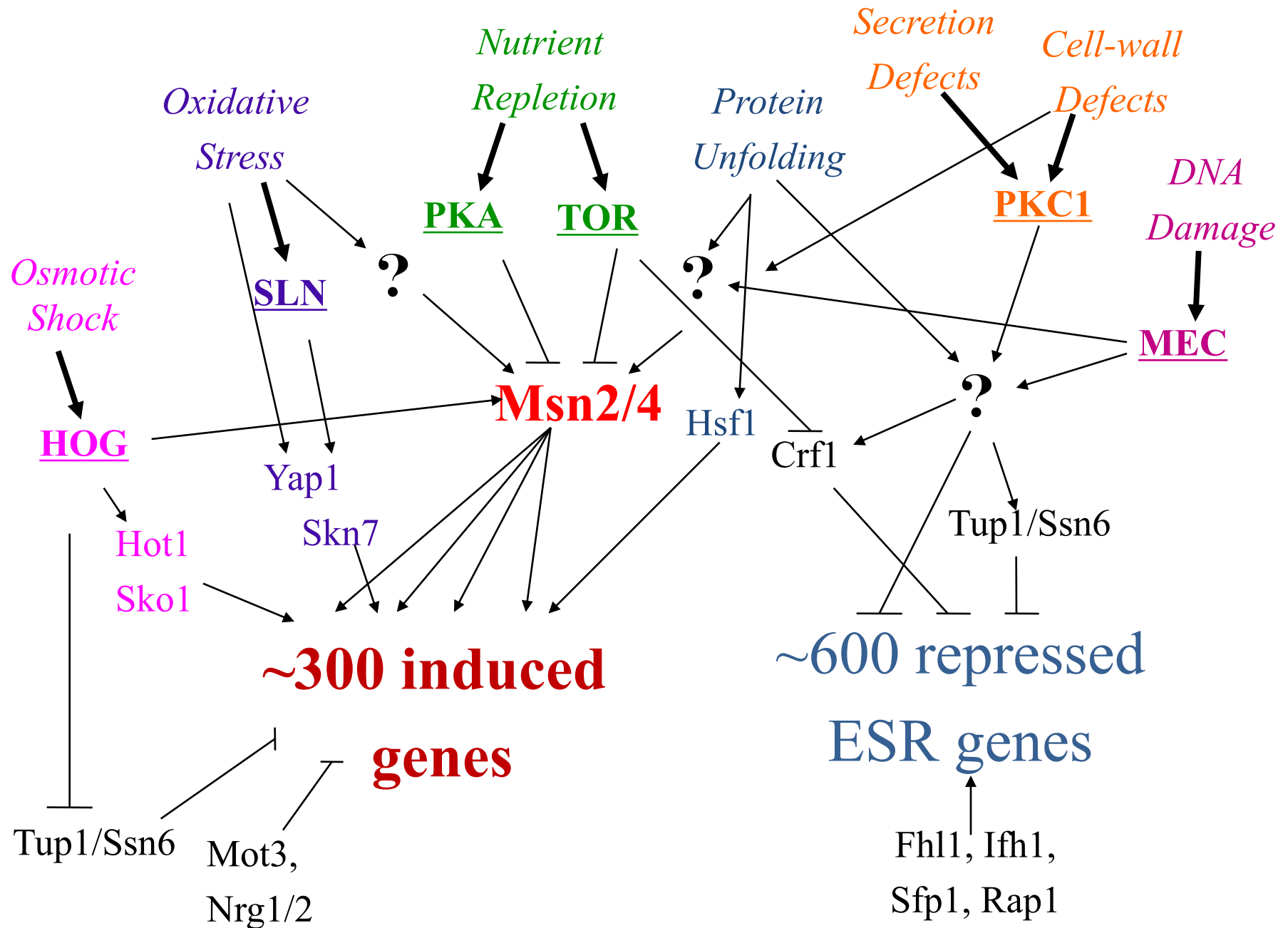
The life history of *Saccharomyces cerevisiae*



The effects of ethanol on the cell



The complexity of stress signaling in yeast



(One of the) motivating
questions for the
generation of the dataset
we will look at:

Is Msn2/4 involved in
regulating the yeast
ethanol response?

YPS606: 5% ethanol stress (30 minutes)

YPS606 *msn2/4*ΔΔ mutant: unstressed

YPS606 *msn2/4*ΔΔ mutant: 5% EtOH stress (30 minutes)

Possible comparisons:

WT unstressed vs WT EtOH stress

*msn2/4*ΔΔ unstressed vs *msn2/4*ΔΔ EtOH stress

WT unstressed vs *msn2/4*ΔΔ unstressed

WT EtOH stress vs *msn2/4*ΔΔ EtOH stress

(WT unstressed vs WT EtOH stress) vs (*msn2/4*ΔΔ unstressed vs *msn2/4*ΔΔ EtOH stress) – comparison of the responses

- e.g., compare 8-fold WT induction by EtOH to 3-fold induction in mutant (defective induction)